

CNC 2020-PSI

March 24, 2021

0.0.1 Les coefficients binomiaux

```
[5]: def fact(n):  
    f=1  
    for i in range(2, n+1):  
        f=f*i  
    return f
```

k : le paramètre de complexité $O(k)$: Complexité linéaire

```
[6]: def binomial(n,k):  
    return fact(n)/(fact(k)*fact(n-k))
```

```
[7]: def binomiaux(n):  
    for i in range(1,n+1):  
        print(binomial(n, i))
```

0.0.2 Calcul numérique

L : Longueur de la barre ==> L = 1.0 D : coefficient de diffusion thermique ==> D = 0.3
Tbarre : Etat initial de la température ==> Tbarre = 100.0 Textr : Température constante
des extrémités ==> Textr = 20.0 T : La durée totale d'évolution ==> T = 1.0 n : nombre
de subdivision ==> n = 10000 dt : T/n p : nombre de tronçons ==> p = 100 dx : L/p

0.0.3 Q1

```
[2]: import numpy as np  
import matplotlib.pyplot as plt  
  
L = 1.0  
D = 0.3  
Tbarre = 100.0  
Textr = 20.0  
T = 1.0  
n = 10000  
dt = T/n  
p = 100  
dx = L/p
```

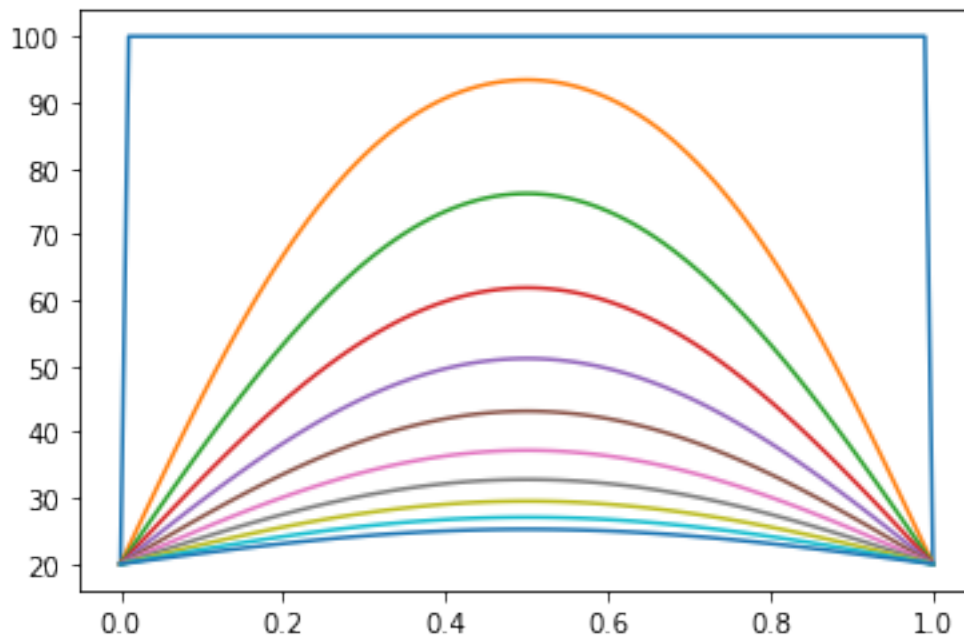
```
def vecteur_init():
    U = np.full(p+1, Tbarre)
    U[0] = Textr
    U[p] = Textr
    return U
```

0.0.4 Q2

```
[3]: def differences_finies(U):
    V = np.zeros(p+1)
    V[0] = U[0]
    V[p] = U[p]
    for i in range(1, len(U)-1):
        V[i] = U[i] + (D * dt/(dx**2)) * (U[i-1]-2*U[i]+U[i+1])
    return V
```

0.0.5 Q3

```
[12]: X=np.linspace(0,L,p+1)
U=vecteur_init()
for i in range (n+1) :
    if i%1000==0: pl.plot(X, U )
    U=differences_finies(U)
```



0.0.6 Base de données et langage SQL

0.0.7 Q1

La clé primaire est composée : (NumC , CodM)

0.0.8 Q2

R = selection(Notes, note>=5 and codM='M') R2 = Jointure(R, Candidats, num=numC) R3 = Projection(R2, num, nom, prénom)

0.0.9 Q3

Select numéro, nom, prénom From Candidats join Notes on numéro=numC Where note>=5 and codM='M'

0.0.10 Q4

Select nom, max(note), min(note) From Matières Join Notes on code=codM Group by code Order by avg(note) Desc

0.0.11 Q5

Select count(distinct numC) From Notes Where note<5

0.0.12 Q6

```
[64]: Select numéro, nom, prénom, sum(note*coefficient) as NT
      From Candidats join (Notes join Matières on code=codM) on numéro=numC
      Where numéro not in (Select numC from Notes where note<5)
      Group by numéro
      Having NT>=1000.0
      Order by NT desc
```

0.0.13 Problème : Transformée de 'Burrows-Wheeler'

0.0.14 Q1

```
[50]: def rotation(T,p):
      return T[p:] + T[:p]
```

0.0.15 Q2

```
[54]: def liste_rotations (T):
      R = []
      for i in range(0, len(T)):
          R.append(rotation(T, i))
      return R
```

0.0.16 Q3

```
[55]: def tri_rotation(R):
    for i in range(0, len(R)-1):
        for j in range(i+1, len(R)):
            if R[i] < R[j] :
                R[i], R[j] = R[j], R[i]
    return R
```

0.0.17 Q4.a

```
[57]: def position(T, R):
    d,f=0,len(T)-1
    while d<=f:
        m=(d+f)//2
        if T==R[m]: return m
        elif T<R[m]: f=m-1
        else: d=m+1
    return -1
```

0.0.18 Q4.b

Complexité quasi-linéaire $O(\text{len}(T)\log(\text{len}(R)))$

0.0.19 Q5

```
[59]: def BWT(T):
    R=liste_rotations(T)
    tri_rotations(R)      #ou   R= tri_rotations(R)
    p=position(T, R)
    B=''
    for e in R:
        B=B+e[-1]
    return (B, p)
```

0.0.20 Q6.a

```
[ ]: def decompression(ch):
    S=""
    for i in range(0,len(ch),2):
        S=S+int(ch[i])*ch[i+1]
    return S
```

0.0.21 Q6.b

```
[60]: def RLE(S):  
    ch = ""  
    i = 0  
    while i<len(S):  
        j=i+1  
        while j<len(S) and S[j]==S[i]:  
            j=j+1  
        c=j-i  
        ch=ch+("9"+S[i])*(c//9)  
        if c%9!=0: ch=ch+str(c%9)+S[i]  
        i=i+c  
    return ch
```